SOFTWARE ENGINEERING LAB – 3

Component Modelling & Architectural Pattern Selection

Problem Statement: Student Project Portfolio & Showcase App (PS4)	**Selected Architectural Style:** Microservices Architecture
Student Name: Porla Neha	**SRN:** PES1UG25AM808
Section: C | Semester: 5	**Functional Requirement:** FR-005 – Project Leaderboard Management

1. Problem Overview

The Student Project Portfolio & Showcase App lets students publish projects with multimedia artifacts (Lab 1 use case: Submit Multimedia Artifacts), judges score them through a rubric, and verified users vote for them. Lab 2 turned FR-005 into the Project Leaderboard Management epic (SCRUM-7, four user stories SCRUM-8 to SCRUM-11). This lab designs the architecture for that functionality.

- **Functional requirements:** rank projects from judge scores and valid public votes (SCRUM-9); display the leaderboard (SCRUM-8); reflect newly submitted valid scores and votes (SCRUM-10); exclude invalid or duplicate votes (SCRUM-11).
- **Non-functional requirements:** NFR-001 – API latency under 300 ms at peak load with authentication and anti-sybil controls; NFR-002 – 99.5% availability and no loss of committed scores or votes.

2. UML Component Diagram

The component diagram below shows the five components of the leaderboard functionality and their labelled interfaces (provided = ball, required = socket).

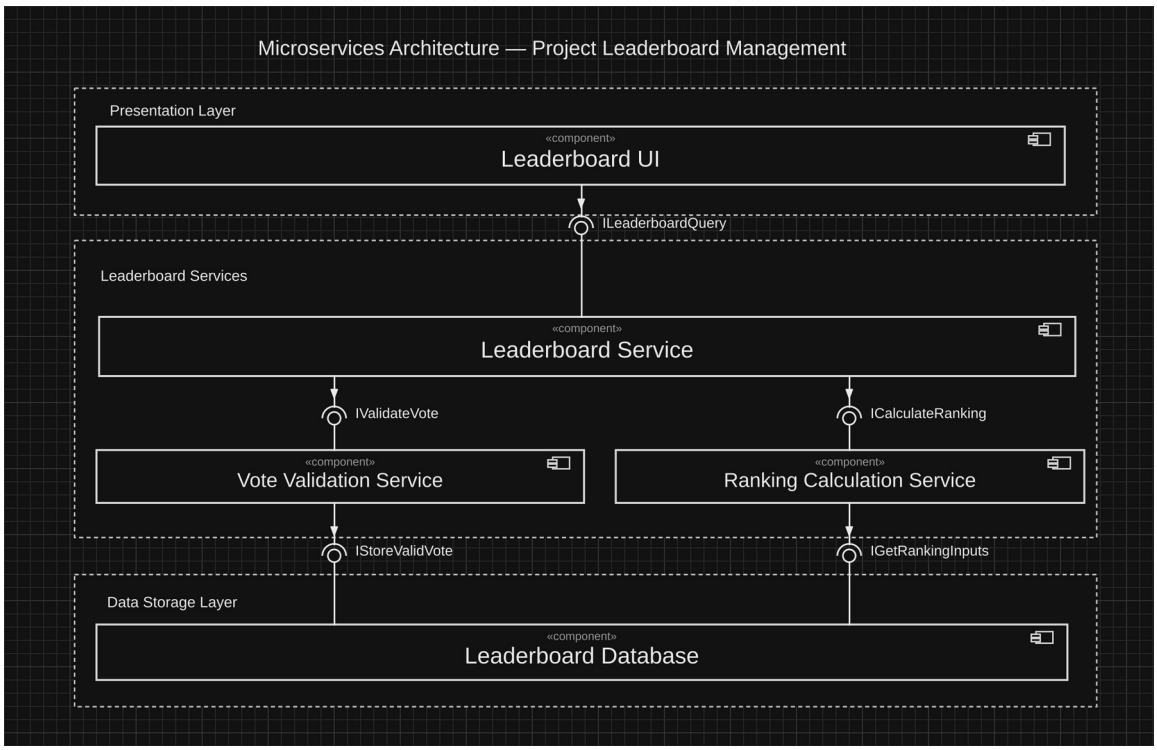


Figure 1. Microservices component diagram – Project Leaderboard Management.

3. Component Responsibilities

- **Leaderboard UI:** displays the leaderboard to public users and sends ranking queries through ILeaderboardQuery (REST / HTTPS) (SCRUM-8).
- **Leaderboard Service:** central coordinator; provides ILeaderboardQuery, returns the latest ranking, and calls IValidateVote and ICalculateRanking (REST / JSON) (SCRUM-8, SCRUM-10).
- **Vote Validation Service:** applies the anti-sybil rules from FR-004, accepts valid votes, rejects invalid or duplicate ones, and stores accepted votes through IStoreValidVote (SCRUM-11).
- **Ranking Calculation Service:** calculates the ranking from judge scores and valid votes, reading its inputs through IGetRankingInputs (SCRUM-9).
- **Leaderboard Database:** persists valid votes, judge scores and ranking inputs; provides IStoreValidVote (SQL write) and IGetRankingInputs (SQL read).

4. Architectural Style Selection and Justification

Microservices Architecture is selected. Layered Architecture is simpler, but the whole application is deployed and scaled as one unit, so a voting spike cannot be scaled on its own. Client-Server places all logic on one central server, which is a single point of failure and conflicts with NFR-002. Two reasons support Microservices: **First,** voting and leaderboard viewing spike during the showcase while judge scoring is low-volume, so Vote Validation and Ranking Calculation can be scaled independently to meet the 300 ms target (NFR-001). **Second,** failures are isolated: if the Ranking Calculation Service fails, valid votes are still validated and stored, and rankings are recalculated from the database on recovery, supporting NFR-002. The trade-off is added network latency and operational complexity, which is accepted for this scope.

5. Security and Performance Considerations

Security advantage: the Vote Validation Service acts as a separate gatekeeper. Only votes that pass the anti-sybil checks are written through IStoreValidVote, so invalid, duplicate or fake votes never reach the ranking inputs, and the UI never accesses the database directly.

Performance benefit: ranking calculation runs in its own service, so heavy computation does not slow down vote validation or leaderboard queries, and it can be scaled separately at peak load.

6. Conclusion

The component model gives the leaderboard functionality (FR-005) five components with defined responsibilities and five labelled interfaces. Microservices Architecture suits it because it supports independent scaling, fault isolation and controlled handling of votes, and it traces back to the Lab 1 requirements and the Lab 2 epic SCRUM-7.